

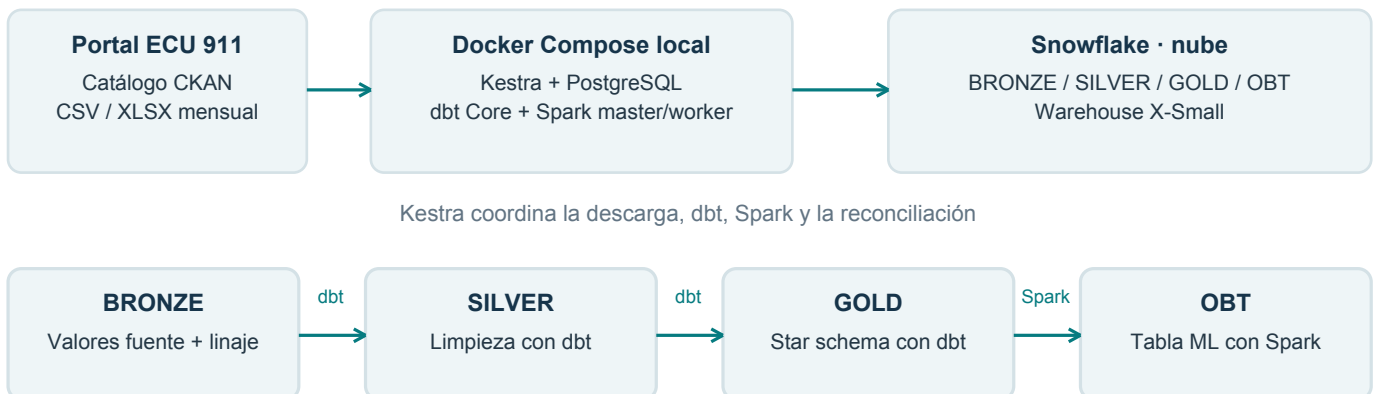
PSet #2 · Pipeline de datos ECU 911

Documento técnico · Ciencia de Datos: end-to-end · 3 de octubre de 2026

Jarod Tierra · Felipe Rodríguez · Santiago Rodríguez
Juan Diego Cadena · Miguel Jarrín

1. Arquitectura y propósito

Implementamos una tubería ELT reproducible que descarga el histórico público de emergencias del ECU 911 y lo prepara en Snowflake para el proyecto del PSet 1: clasificar demanda inusualmente alta por cantón a **t+3** y **t+7**. Este entregable termina en datos preparados; no entrena ni evalúa el modelo predictivo.



Responsabilidades. Kestra descubre y descarga los archivos, carga Bronze y orquesta las transformaciones. dbt construye Silver y el star schema Gold dentro de Snowflake. Spark lee Gold, construye la OBT y la escribe nuevamente en Snowflake. PostgreSQL almacena únicamente la metadata de Kestra; los datos analíticos residen en Snowflake.

Infraestructura. Docker Compose define Kestra 1.3.40, PostgreSQL 16, Spark 3.5.9 con un master y un worker, y la preparación de dbt Core/Snowflake 1.9.0. La imagen de dbt se construye para la arquitectura del equipo. El servicio kestra-init importa los siete flows y sube el proyecto dbt y el job Spark a los archivos del namespace ecu911; termina sin ejecutar la ingesta.

Resultado verificado: 62 meses (julio de 2021–agosto de 2026), 17.680.253 registros fuente y una OBT de 422.912 filas × 37 columnas. La corrida desde cero se verificó el 2 de octubre de 2026.

Configuración y acceso. .env.example define las variables; .env contiene las credenciales y está excluido de Git. El setup se conecta con las credenciales SNOWFLAKE_ADMIN_USER/PASSWORD y el rol ACCOUNTADMIN para crear los objetos y configurar permisos. Asigna SNOWFLAKE_ROLE al usuario SNOWFLAKE_USER; las cargas y transformaciones utilizan esos valores del .env. El warehouse es X-Small, con auto-suspensión de 60 segundos.

Repositorio del proyecto. https://github.com/junilai/Pset_2

2. Ingesta, programación y recuperación

Fuente y grain original. El dataset «ECU911 Base de Emergencias» publica un archivo mensual con siete campos: fecha, provincia, cantón, código de parroquia, parroquia, servicio y subtipo. Cada fila es un registro de emergencia de la fuente, sin identificador de incidente. En el histórico verificado hay 60 CSV separados por punto y coma y dos XLSX (abril y mayo de 2026).

Descubrimiento y carga automática

El flow `ingest_month` recibe un mes `YYYY-MM` y consulta la API CKAN `package_show` del dataset `base-de-emergencias`. Busca `YYYYMM` en el nombre de sus recursos y obtiene la URL publicada; no mantiene una lista manual de enlaces. Si el recurso no existe, falla con un mensaje explícito. Descarga el archivo y convierte los XLSX a CSV preservando valores; las fechas seriales se interpretan posteriormente en Silver.

El archivo se sube a un stage interno de Snowflake con una ruta exclusiva por mes y ejecución. Bronze conserva los siete campos como texto y añade periodo, URL de origen, nombre y fila del archivo, fecha de carga e ID de ejecución de Kestra. La conversión XLSX cambia el contenedor del archivo, no aplica reglas de limpieza. Los archivos temporales se purgan tras la ejecución.

Idempotencia, retries y propagación de errores

La carga ejecuta **BEGIN** → **DELETE del periodo** → **COPY INTO con FORCE=TRUE** → **COMMIT**. Repetir un mes reemplaza su partición en vez de añadir otra copia. El ID técnico de Silver combina periodo y fila del archivo; identifica la fila cargada, no demuestra la identidad del incidente real. Se verifica que la carga no quede vacía.

La consulta del catálogo y la descarga tienen retry exponencial: intervalo inicial de 30 segundos, máximo de 5 minutos y `maxAttempts=4`. La subida y carga usan retry constante de 30 segundos con `maxAttempts=3`. `ingest_month` limita la concurrencia a cuatro ejecuciones y encola las restantes. Un fallo de dbt detiene Spark; una validación fallida de Spark impide escribir la OBT.

Frecuencia y backfill

Recorrido	Implementación
Programación	<code>ingest_scheduled</code> : día 10, 06:00, America/Guayaquil (cron 0 6 10 * *). Recarga los dos meses anteriores y lanza transform; recupera fechas de programación perdidas.
Histórico por rango	<code>run_all</code> : setup opcional → <code>ingest_month</code> por cada mes (hasta cuatro a la vez) → una sola transformación final. Rango reproducible: 2021-07 a 2026-08.
Backfill del trigger	Backfill executions de monthly, desde agosto de 2021: <code>meses=1</code> y <code>transformar=false</code> . Cada fecha carga su mes anterior; después se ejecuta transform una vez.
Recuperar un mes	<code>run_all</code> con <code>desde=hasta=mes requerido</code> y <code>setup=false</code> . Recarga Bronze y reconstruye Silver, Gold y OBT sobre todo el histórico disponible.

Estado WARNING. Al agotarse los retries, el subflow del mes falla y la ejecución padre queda en WARNING; los demás meses continúan y después se transforman los datos disponibles. Los tests bloquean huecos interiores, mientras que un último mes aún no publicado no genera datos ficticios.

3. Calidad: evidencia y decisiones

El perfilado previo a la limpieza revisó **completitud, validez, consistencia y precisión** sobre 17.680.253 filas. Las consultas están en docs/data_quality.sql (29 de septiembre de 2026); las métricas finales de capas se contrastaron en la corrida del 2 de octubre. Todas las reglas de limpieza se implementan con dbt en Silver.

Problema	Evidencia	Acción	Justificación
Completitud: ubicación	2.127 ubicaciones vacías, 17 con texto NULL y 16 con 0: 2.160 filas (0,012%).	Normalizar a NULL y marcar es_ubicacion_valida=false. Conservar en Silver; excluir del agregado geográfico.	No se puede asignar un cantón sin inventar información; conservar la fila mantiene la trazabilidad.
Validez del código DPA	1.265.543 códigos de cinco dígitos (7,16%), en diez meses.	LPAD a seis dígitos para los códigos numéricos válidos.	Recupera el cero inicial perdido sin cambiar el significado del código.
Consistencia código/nombre	44 filas de Morona Santiago con código de parroquia inconsistente 090150.	Asignar el código de cantón más frecuente para cada grupo de provincia y cantón.	La agrupación provincial evita fusionar homónimos como Bolívar y Olmedo. La clave se infiere del resto de los registros.
Consistencia de texto	4.637 filas con LOGRODO; 234 con LOGROÑO.	Sustituir Đ por Ñ y aplicar TRIM en textos.	Unifica una variante de codificación del mismo cantón, código 1410.
Validez de fechas	559.737 fechas seriales de Excel; cero fechas no interpretables y cero fuera de su mes.	Interpretar DD/MM/YYYY o serial Excel y convertir a DATE.	Ambos formatos representan fechas; rechazarlos perdería meses válidos.
Identidad de incidentes	11.747.431 repeticiones de los siete atributos (66,44%) según el perfilado.	Conservar; no deduplicar por atributos.	Sin ID no se puede confirmar que sean el mismo incidente. Eliminarlas podría subestimar la demanda.
Precisión del volumen	Enero de 2024: 208.194 filas; volumen/día 28,2% bajo la mediana de vecinos.	Conservar y marcar es_periodo_anomalo; excepción explícita al test de estabilidad.	La causa no está confirmada. La etapa ML debe evaluar su tratamiento, sin ocultar el periodo.
Comparabilidad geográfica	11.019 filas históricas de la parroquia Sevilla Don Bosco reasignadas desde Morona.	Aplicar el seed cantones_reasignados a la geografía vigente.	Evita un quiebre artificial en ambas series. Es una decisión de modelado, no geografía histórica versionada.

Asignación del código cantonal. La regla se aplica a todos los grupos de provincia y cantón. Compara los primeros cuatro dígitos del código de parroquia normalizado con el código cantonal más frecuente del grupo; si difieren, marca es_codigo_corregido=true. No contrasta con un catálogo oficial. La reasignación de Sevilla Don Bosco se aplica después mediante el seed.

Completitud temporal. Los tests comprueban días completos por mes y ausencia de meses interiores faltantes. Gold incorpora 26.716 cantón-días sin registros (6,32%) como cero dentro del rango cubierto. No extiende esos ceros a meses posteriores aún no publicados.

Alcance de la precisión. Las anomalías y las reglas de formato permiten detectar incoherencias; no prueban que cada incidente haya ocurrido como está registrado. No hay una fuente independiente ni ID de incidente para medir exactitud factual o confirmar duplicados.

4. Transformaciones y star schema

Bronze → Silver. source() declara BRONZE.EMERGENCIAS_RAW. El modelo emergencias interpreta fechas, normaliza textos/códigos, resuelve claves geográficas y añade indicadores de calidad. ref() conecta el seed de reasignación y los modelos derivados. Silver mantiene una fila por registro fuente: **17.680.253 filas**, incluidas las 2.160 no asignables geográficamente.

Silver → Gold. El hecho cuenta solo las filas con ubicación válida por (cod_canton, fecha). Un producto de todas las unidades geográficas y fechas del rango genera el date spine; un LEFT JOIN con los conteos rellena con cero los días sin registros. Las claves naturales evitan unir cantones homónimos y simplifican la conexión con Spark.

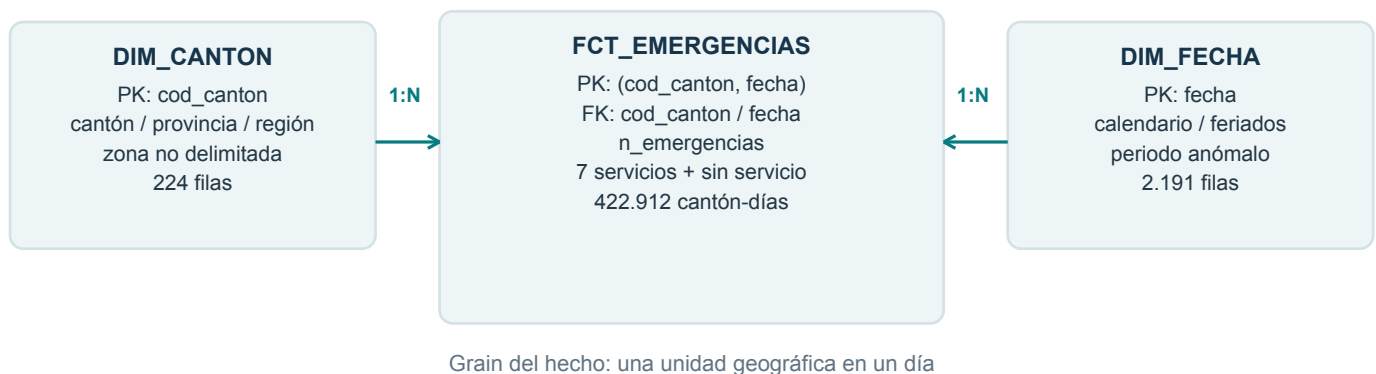


Tabla	Grain y contenido	Filas
DIM_CANTON	Una unidad geográfica por código DPA; cantón, provincia, región y bandera de zona no delimitada.	224
DIM_FECHA	Un día calendario de 2021 a 2026; fecha, día de semana, mes, fin de semana, feriado y periodo anómalo.	2.191
FCT_EMERGENCIAS_CANTON_DIA	Una unidad geográfica × día; total, siete conteos por servicio y conteo sin servicio. PK compuesta (cod_canton, fecha).	422.912

Las 224 unidades incluyen Sevilla Don Bosco, Las Golondrinas y El Piedrero de zona no delimitada. Usamos geografía vigente sin historia SCD. Los servicios se mantienen como columnas porque el conjunto es fijo y ML requiere una fila por cantón-día. Los feriados son un seed versionado. **DIM_FECHA cubre de forma fija 2021-01-01 a 2026-12-31**, suficiente para consultar t+3/t+7 en el histórico documentado hasta 2026-08. Para procesar periodos posteriores debe ampliarse junto con el seed de feriados.

Tests que protegen reglas del proyecto

dbt valida not_null y unique en claves, relationships del hecho hacia las dimensiones y unicidad del grain compuesto. Los tests singulares comprueban conservación Bronze–Silver, fechas dentro de su archivo mensual, meses/días completos, cobertura cantonal, spine completo, suma de servicios y conservación de emergencias. La tolerancia de filas sin ubicación es 0,1%; el volumen diario mensual debe quedar a ±20% de la mediana de vecinos, salvo el periodo anómalo declarado.

Reconciliación: $224 \times 1.888 = 422.912$ cantón-días.

$SUM(n_emergencias) = 17.680.253 - 2.160 = 17.678.093$.

dbt build aprobó 33 tests, cuatro modelos y dos seeds: PASS=39, WARN=0, ERROR=0, SKIP=0.

5. Spark, OBT y preparación del target

build_obt.py lee el hecho y las dos dimensiones Gold mediante el conector Spark–Snowflake. Une las dimensiones con LEFT JOIN y broadcast, dado su tamaño reducido. Conserva el grain **una unidad geográfica × día** y añade variables temporales mediante ventanas por cantón ordenadas por fecha. Los feriados a $t+3/t+7$ se consultan en dim_fecha, incluso cuando aún no existen conteos futuros.

Grupo	Columnas / uso
Identificación y contexto	cod_canton, fecha, provincia, región y atributos del calendario.
Volumen observado en t	n_emergencias y conteos por servicio, disponibles al cierre de t en la convención del histórico.
Historial previo	lag_1d, lag_7d, lag_14d, lag_28d; media_prev_7d y media_prev_28d. Las medias excluyen t y requieren la ventana completa.
Futuro conocido	es_feriado_t3 y es_feriado_t7: información de calendario.
Insumos de etiqueta	obj_n_emergencias_t3/t7: conteos reales futuros. Se usan para construir el target y se excluyen de las variables predictoras.

Alta demanda: clasificación, no predicción del conteo

El PSet 1 define $y_h=1$ cuando el volumen del cantón en $t+h$ supera su P90 histórico para el **día de la semana de $t+h$** ; en caso contrario $y_h=0$, con $h = 3$ o $h = 7$. El P90 se calculará únicamente con el periodo de entrenamiento. La OBT contiene los conteos futuros observados necesarios para generar la etiqueta, pero no calcula todavía el percentil ni las clases.

Ejemplo ilustrativo: si el P90 de entrenamiento de un cantón para el día objetivo es 100 y $obj_n_emergencias_t3=120$, $y_3=1$ (alta). Si el conteo es 80, $y_3=0$ (no alta). Estos valores ilustran la regla; no son un resultado del modelo.

Validación de joins y ventanas

Antes de escribir se exigen siete comprobaciones: filas OBT=hecho, claves distintas=filas, suma de emergencias conservada, cero filas sin match de cantón, cero sin match de fecha y cero nulos en cada feriado futuro ($t+3$ y $t+7$). Si alguna falla, el job termina sin escribir. La salida se guarda con overwrite en OBT.OBT_EMERGENCIAS_CANTON_DIA y se relee el conteo para verificar la escritura.

Resultados: 422.912 filas, 422.912 claves distintas, 37 columnas y 17.678.093 emergencias. SQL independiente confirmó cero discrepancias en lag_28d y conteos objetivo futuros, y cero en medias a los cuatro decimales almacenados. Los nulos iniciales son esperados: $lag_28d/media_prev_28d=6.272$ y $media_prev_7d=1.568$. Los objetivos finales quedan nulos cuando no existe observación futura; no se imputan como cero.

Uso de cada estructura. Gold es apropiado para consultas y reportes por cantón, fecha o servicio, manteniendo dimensiones reutilizables. La OBT reúne contexto y variables en una tabla para entrenamiento y evaluación temporal, evitando repetir joins. Al preparar ML se excluirán las columnas obj_* de las entradas y se tratarán las ventanas/etiquetas incompletas.

6. Batch vs. streaming

Frecuencia de los datos. Las emergencias ocurren durante el día, pero la fuente pública utilizada entrega archivos consolidados por mes. Por eso implementamos batch: Kestra carga meses completos el día 10 y revisa los dos meses anteriores para recuperar publicaciones tardías. Este ritmo es suficiente para actualizar el histórico de entrenamiento y evaluación del PSet 2; procesar continuamente no adelantaría la publicación de la fuente.

Latencia del negocio. El caso del PSet 1 busca anticipar demanda alta a tres y siete días para revisar turnos y planificación semanal. Esa decisión puede apoyarse en una actualización diaria, sin exigir respuesta en segundos. Con datos diarios oportunos, un batch al cierre de cada jornada sería una opción suficiente para ese ciclo de revisión. La fuente mensual actual sirve para preparar y evaluar el histórico, pero su retraso todavía limita las alertas operativas a esos horizontes.

Cambio que justificaría streaming. Se justificaría si el caso de uso pasara a detectar aumentos de demanda o riesgo de saturación durante la jornada y actualizar alertas en minutos. Además de esa nueva exigencia de latencia, necesitaríamos acceso a un flujo continuo de incidentes con identificadores y marcas de tiempo. La arquitectura incorporaría captura incremental y procesamiento de eventos, con control de duplicados y llegadas tardías. Para el alcance actual se mantiene batch; no se implementa streaming.

Verificación y reproducción

La prueba del **2 de octubre de 2026** creó los objetos y cargó 62 meses desde cero. La corrida completa duró **6 min 18 s**: WARNING por septiembre no publicado, con transformación SUCCESS. Recargar agosto y reconstruir las capas tomó **3 min 4 s**, con SUCCESS, iguales conteos y cero claves OBT duplicadas. Repetir el setup funcionó y un rango invertido fue rechazado antes de cargar.

Para reproducir: copiar .env.example a .env, completar credenciales y ejecutar **docker compose up -d**. Revisar dbt y kestra-init con Exited (0); abrir Kestra en localhost:8080 y ejecutar `ecu911.run_all` con **desde=2021-07, hasta=2026-08, setup=true**. Revisar los subflows y la reconciliación. No se necesitan cargas manuales. Las duraciones corresponden al entorno probado, no son garantías de rendimiento.

7. Limitaciones

Disponibilidad temporal. El histórico público permite preparación y evaluación retrospectiva, pero su retraso mensual impide asumir que `n_emergencias` o los lags estaban disponibles al cierre real de cada día. Un uso operativo debe incorporar datos oportunos y validar tiempos de disponibilidad; el PSet 2 no resuelve esa integración.

Fuentes y calidad. La población cantonal INEC prevista en el PSet 1 no está integrada; no hay tasas por habitante. Enero de 2024 tiene causa de anomalía sin confirmar. Sin ID ni fuente de contraste no se garantiza exactitud factual ni deduplicación de incidentes. Los ceros indican ausencia de registros en periodos con cobertura validada, no una auditoría externa de ausencia de emergencias.

Correcciones de meses antiguos. El schedule recarga únicamente los dos meses anteriores. Si el portal modifica un archivo más antiguo, el cambio no se incorpora automáticamente: se debe recargar ese mes con `run_all` (`desde=hasta=mes, setup=false`) o un backfill y reconstruir las capas derivadas.

Mantenimiento y arquitectura. `dim_fecha` y los feriados cubren hasta diciembre de 2026 y requieren ampliación. Silver/Gold y OBT se reconstruyen completas; no hay transformación incremental. La reproducción depende de permisos de Snowflake, recursos de Docker y disponibilidad del portal y de Snowflake.